

agent.py

```
import imaplib

import email

from email.header import decode_header

from config import load_institutes, CONFIDENCE_THRESHOLD,
get_credentials

from email_service import EmailService

from ai_classifier import AIClassifier

from audit_logger import AuditLogger


class MailClerkAgent:

    def __init__(self):

        # Retrieve credentials from Secrets Manager/env vars - single source
        # of truth, used by both EmailService and this class's own IMAP fallback.
        self.creds = get_credentials()

        self.email_username = self.creds.get("EMAIL_USERNAME")
        self.app_password = self.creds.get("EMAIL_PASSWORD")
        self.imap_server = self.creds.get("IMAP_SERVER", "imap.gmail.com")


        self.email_service = EmailService(self.creds)

        self.classifier = AIClassifier()

        self.logger = AuditLogger()

        self.institutes = load_institutes()
```

```

def classify_text(self, text_content):
    """Processes and classifies direct string payloads (e.g., from Streamlit)."""
    classification = self.classifier.classify_email("Streamlit Direct Test",
text_content, self.institutes)

    target = next((i for i in self.institutes if i["institute_id"] ==
classification.get("institute_id")), None)

    target_name = target["name"] if target else "Finance & Billing Department"

    return {
        "id": "MSG_DIRECT_101",
        "status": "FORWARDED",
        "target": target_name,
        "confidence": classification.get("confidence", 0.95),
        "summary": text_content
    }

```

```

def fetch_unread_via_imap(self):
    """Connects directly to Gmail via IMAP to pull unread incoming emails."""
    fetched_emails = []

    if not self.email_username or not self.app_password:
        print("IMAP fetch skipped: EMAIL_USERNAME/EMAIL_PASSWORD
not set (check Lambda environment variables or Secrets Manager).")
        return fetched_emails

    try:
        mail = imaplib.IMAP4_SSL(self.imap_server)

```

```

mail.login(self.email_username, self.app_password)

mail.select("inbox")

# Search for unread emails (or filter by specific sender e.g.,
janethra127@gmail.com)

status, messages = mail.search(None, 'UNSEEN')
email_ids = messages[0].split()

for e_id in email_ids:

    status, msg_data = mail.fetch(e_id, '(RFC822)')

    for response_part in msg_data:

        if isinstance(response_part, tuple):

            msg = email.message_from_bytes(response_part[1])

            # Extract Subject

            subject = "No Subject"

            if msg["Subject"]:

                decoded_subject, encoding = decode_header(msg["Subject"])[0]

                if isinstance(decoded_subject, bytes):

                    subject = decoded_subject.decode(encoding or "utf-8",
errors="ignore")

                else:

                    subject = str(decoded_subject)

            # Extract Body Text

```

```

        body = ""
        if msg.is_multipart():
            for part in msg.walk():
                content_type = part.get_content_type()
                content_disposition = str(part.get("Content-Disposition"))
                if content_type == "text/plain" and "attachment" not in
content_disposition:
                    body =
part.get_payload(decode=True).decode(errors="ignore")
                    break
            else:
                body = msg.get_payload(decode=True).decode(errors="ignore")

        fetched_emails.append({
            "id": e_id.decode("utf-8"),
            "subject": subject,
            "body": body
        })

    mail.logout()
except Exception as e:
    print(f'IMAP Fetching Error: {str(e)}')

return fetched_emails

```

```

def process_incoming_emails(self):
    """Processes unread emails using EmailService or IMAP fallback."""
    # Attempt to fetch emails via primary EmailService, fallback to IMAP if
    empty
    emails = self.email_service.fetch_unread_emails() if
hasattr(self.email_service, "fetch_unread_emails") else []
    if not emails:
        emails = self.fetch_unread_via_imap()

    results = []
    for email_item in emails:
        classification = self.classifier.classify_email(
            email_item.get("subject", ""),
            email_item.get("body", ""),
            self.institutes
        )

        if classification.get("confidence", 0) >= CONFIDENCE_THRESHOLD:
            target = next((i for i in self.institutes if i["institute_id"] ==
classification.get("institute_id")), None)

            target_email = target["contact_email"] if target else
"kalugampitige2@gmail.com"

            target_name = target["name"] if target else "Finance & Billing
Department"

            forward_ok = True

```

```

forward_error = None
if hasattr(self.email_service, "forward_email"):
    try:
        self.email_service.forward_email(email_item["id"], target_email)
    except Exception as forward_err:
        # Don't let one bad/unverified address wipe out this
        # email's real subject/body - log it and keep going.
        forward_ok = False
        forward_error = str(forward_err)
        self.logger.log_action(email_item["id"], "FORWARD", "FAILED",
forward_error)

        self.logger.log_action(email_item["id"], "FORWARD", "SUCCESS" if
forward_ok else "FAILED", target_email)
    results.append({
        "id": email_item["id"],
        "status": "FORWARDED" if forward_ok else "FORWARD_FAILED",
        "target": target_name,
        "target_email": target_email,
        "error": forward_error,
        "summary": f"Subject: {email_item.get('subject')}\nBody:
{email_item.get('body')}"
    })
else:

```

```

        self.logger.log_action(email_item["id"], "FLAG_HUMAN_REVIEW",
                                "LOW_CONFIDENCE", classification)

        results.append({
            "id": email_item["id"],
            "status": "FLAGGED",
            "target": "Needs Human Review",
            "reason": "Low confidence",
            "confidence": classification.get("confidence", 0),
            "summary": f"Subject: {email_item.get('subject')}\nBody:
{email_item.get('body')}"
        })

```

```

    return results

```

```

if __name__ == "__main__":
    agent = MailClerkAgent()
    print(agent.process_incoming_emails())

```

ai_classifier.py

```

import json
import boto3
from config import AWS_REGION, CONFIDENCE_THRESHOLD

```

```

class AIClassifier:

    def __init__(self):
        self.bedrock = boto3.client("bedrock-runtime",
region_name=AWS_REGION)

    def match_by_keyword(self, subject, body, institutes):
        """Checks institutes.json's keyword lists against the email text.
Runs before the LLM call - cheaper, and gives predictable routing
for anything with an obvious keyword match."""
        text = f'{subject} {body}'.lower()
        for inst in institutes:
            for kw in inst.get("keywords", []):
                if kw.lower() in text:
                    return {"institute_id": inst["institute_id"], "confidence": 0.90}
        return None

    def classify_email(self, subject, body, institutes):
        keyword_match = self.match_by_keyword(subject, body, institutes)
        if keyword_match:
            return keyword_match

        institute_ids = [i["institute_id"] for i in institutes]
        prompt = (
            f'Categorize this email into exactly one of these IDs: {institute_ids}.\n'
            f'Subject: {subject}\nBody: {body}\n\n'

```

```

'Respond with ONLY raw JSON, no other text: '
'{"institute_id": "<one of the IDs above>", "confidence": <0.0 to 1.0>}'
)
try:
    payload = {
        "inferenceConfig": {"max_new_tokens": 300},
        "messages": [{"role": "user", "content": [{"text": prompt}]}]
    }
    response = self.bedrock.invoke_model(
        modelId="us.amazon.nova-pro-v1:0",
        body=json.dumps(payload)
    )
    response_body = json.loads(response["body"].read())
    raw_text =
response_body["output"]["message"]["content"][0]["text"].strip()
    raw_text = raw_text.strip("").replace("json", "", 1) if
raw_text.startswith("```") else raw_text
    result = json.loads(raw_text)
    if result.get("institute_id") in institute_ids:
        return {"institute_id": result["institute_id"], "confidence":
float(result.get("confidence", 0.5))}
    raise ValueError(f"Model returned unknown institute_id:
{result.get('institute_id')}")
except Exception as e:
    print(f"Bedrock classification failed, using keyword fallback: {e}")

```

```
# Fallback heuristic rule for testing mode / Bedrock unavailable

if "invoice" in body.lower():

    return {"institute_id": "INST_FINANCE", "confidence": 0.90}

return {"institute_id": "INST_HR", "confidence": 0.80}
```

audit_logger.py

```
import logging

logging.basicConfig(level=logging.INFO, format="%asctime)s - %(levelname)s - %(message)s")

class AuditLogger:

    def log_action(self, email_id, action, status, details=None):

        logging.info(f'Email ID: {email_id} | Action: {action} | Status: {status} | Details: {details}')
```

config.py

```
import os

import json

import boto3

try:

    from dotenv import load_dotenv
```

```

load_dotenv()
except ImportError:
    pass

AWS_REGION = os.getenv("AWS_REGION", "us-east-1")
SECRET_NAME = os.getenv("SECRET_NAME", "mail-clerk/credentials")
CONFIDENCE_THRESHOLD =
float(os.getenv("CONFIDENCE_THRESHOLD", "0.85"))
TEST_MODE = os.getenv("TEST_MODE", "true").lower() == "true"

def load_institutes(filepath="institutes.json"):
    """Loads department/institute routing configurations from a local JSON file."""
    try:
        with open(filepath, "r") as f:
            return json.load(f)
    except Exception as e:
        print(f"Warning: Could not load {filepath}, error: {e}")
        return []

def get_credentials():
    """Retrieves secret credentials from AWS Secrets Manager or falls back to local
    environment variables."""
    try:
        client = boto3.client("secretsmanager", region_name=AWS_REGION)
        secret_value = client.get_secret_value(SecretId=SECRET_NAME)

```

```

        return json.loads(secret_value["SecretString"])
    except Exception as e:
        print(f'AWS Secrets Manager retrieval failed ({e}), falling back to local .env
variables...")
    return {
        "EMAIL_USERNAME": os.getenv("EMAIL_USERNAME", ""),
        "EMAIL_PASSWORD": os.getenv("EMAIL_PASSWORD", ""),
        "IMAP_SERVER": os.getenv("IMAP_SERVER", "imap.gmail.com"),
        "SMTP_SERVER": os.getenv("SMTP_SERVER", "smtp.gmail.com")
    }

```

email_service.py

```

import boto3
import imaplib
import email
from email.mime.text import MIMEText
from botocore.exceptions import ClientError

class EmailService:
    def __init__(self, credentials):
        self.creds = credentials or {}
        self.ses_client = boto3.client('ses', region_name='us-east-1')
        self.verified_sender = "kalugampitige@gmail.com"

```

```

def fetch_unread_emails(self):
    """Fetches unread emails from Gmail IMAP."""
    username = self.creds.get("EMAIL_USERNAME")
    password = self.creds.get("EMAIL_PASSWORD")
    imap_server = self.creds.get("IMAP_SERVER", "imap.gmail.com")

    if not username or not password:
        print("Error: EMAIL_USERNAME or EMAIL_PASSWORD missing in
credentials.")
        return []

    try:
        mail = imaplib.IMAP4_SSL(imap_server)
        mail.login(username, password)
        mail.select("INBOX")

        status, messages = mail.search(None, "UNSEEN")
        email_list = []

        if status == "OK" and messages[0]:
            for num in messages[0].split():
                _, data = mail.fetch(num, "(RFC822)")
                raw_email = data[0][1]
                msg = email.message_from_bytes(raw_email)

```

```

        body = ""
        if msg.is_multipart():
            for part in msg.walk():
                if part.get_content_type() == "text/plain":
                    body =
part.get_payload(decode=True).decode(errors="ignore")
                    break
        else:
            body = msg.get_payload(decode=True).decode(errors="ignore")

        email_list.append({
            "id": num.decode(),
            "subject": msg.get("Subject", "(No Subject)"),
            "sender": msg.get("From", ""),
            "body": body
        })

    mail.logout()

    return email_list

except Exception as e:
    print(f'Error fetching emails via IMAP: {e}')
    return []

def fetch_todays_inbox(self):

```

```

return self.fetch_unread_emails()

def forward_email(self, email_id, target_email):
    """Sends forward message via Amazon SES."""
    try:
        return self.ses_client.send_email(
            Source=self.verified_sender,
            Destination={'ToAddresses': [target_email]},
            Message={
                'Subject': {'Data': f'Fwd: Email ID {email_id}'},
                'Body': {'Text': {'Data': f'Forwarded automatically by AI Mail Clerk
for Email ID: {email_id}'}}}
            )
    except ClientError as e:
        print(f'Error forwarding email via SES: {e.response['Error']['Message']}")
        raise e

```

institutes.json

```

[
  {
    "institute_id": "INST_FINANCE",
    "name": "Finance & Billing Department",

```

```
"contact_email": "kalugampitige2@gmail.com",
"keywords": ["invoice", "payment", "receipt", "billing", "reimbursement"]
},
{
  "institute_id": "INST_HR",
  "name": "Human Resources",
  "contact_email": "janethra127@gmail.com",
  "keywords": ["payroll", "vacation", "onboarding", "benefits", "policy"]
},
{
  "institute_id": "INST_IT_SUPPORT",
  "name": "IT Helpdesk",
  "contact_email": "kalugampitige2@gmail.com",
  "keywords": ["password", "hardware", "software", "access", "bug"]
}
]
```

Instruction.md

Job Role: act as a first-line triage and routing clerk for incoming institutional email . this is the job that a human mail clerk would traditionally do: open each piece of mail, figure out which department or institute it's actually intended for, and get it there, without every email needing someone to manually read and forward it.

Phase 1 — User Prompt Trigger

****Purpose:**** Start a run.

****Instructions:****

- Accept a run request from one of three sources: a scheduled job (default, e.g. hourly), an on-demand prompt or button press, or (optionally, later) a webhook on new mail.
- On trigger, resolve "today" using the mailbox owner's local timezone, not server time or UTC — build the date range for Phase 2 from that.
- Each run gets a unique run ID, used to group every email processed in that run for later auditing.
- Do not start a second run for the same schedule slot if a prior run for it is still in progress — treat this as a re-entrancy guard, not just an idempotency check at the email level (Phase 2 handles that separately).

Phase 2 — Login & Fetch Mail

****Purpose:**** Securely connect to the mailbox and retrieve only what's new and relevant.

****Instructions:****

- Authenticate using the narrowest scope available: app-only/service auth for a shared institutional mailbox (preferred), OAuth delegated

auth for a personal mailbox, or IMAP/app-password for the fastest prototype path.

- Refresh access tokens proactively before expiry; treat an auth failure as its own alert category, separate from a classification failure.
 - Query mail using the resolved local-date range from Phase 1. Paginate fully — do not stop after the first page of results.
 - Cross-check every fetched message ID against the processed-IDs store before doing any further work on it. Skip anything already processed.
- This is what makes repeated or overlapping runs safe.

Phase 3 — Classify per Email

****Purpose:**** Determine which institute an email belongs to, using the strongest available evidence first.

****Instructions:****

- Evaluate signals in this order, stopping at the first confident match:
 1. ****Sender domain / address**** — check against a maintained lookup table of known institute domains. Strongest signal; treat as near-certain when it hits.
 2. ****Explicit mention**** — institute name, program name, or a reference/case number in the subject or body.
 3. ****Thread history**** — if this is a reply in a thread already routed to an institute, inherit that routing rather than re-deciding.

4. ****Semantic match (LLM)**** — only when the above signals are absent, ask the model to classify against the list of known institutes and return a confidence score.
- If an email plausibly matches more than one institute with similar scores, treat that ambiguity itself as a reason to lower confidence — do not force a single pick.
 - Record which signal produced the match, not just the final answer — this is required for Phase 6's audit trail.

Phase 4 — Confidence Check

****Purpose:**** Decide whether the match from Phase 3 is trustworthy enough to act on automatically.

****Instructions:****

- Compare the match's confidence score against the configured threshold.
- Do not use a single global threshold for every signal type: a domain-match should clear the bar more easily than a semantic-only match, since the underlying evidence quality differs.
- Set the threshold conservatively at first (biased toward flagging, not auto-sending), since a wrong auto-send is more costly than an unnecessary human review.
- Re-derive the threshold periodically from labeled data and live audit outcomes — never hand-tune it off a handful of recent misses.

Phase 5a — Auto-Send (high confidence)

****Purpose:**** Route the email to the matched institute without human intervention.

****Instructions:****

- Re-check the processed-IDs store immediately before sending, as a final idempotency guard.
- Send to the institute's contact address from your own verified mapping table — never an address parsed out of the email body itself.
- Mark the message as processed atomically, immediately after a successful send — not in a batch at the end of the run.
- On a send failure (bounce, API error, rate limit), do not retry blindly — route the email to Phase 5b (flag for review) instead, and record the failure explicitly.
- Watch for run-level anomalies (e.g. an unusual volume spike to one institute) and raise an alert independent of the per-email logic.

Phase 5b — Flag for Review (low confidence)

****Purpose:**** Hand uncertain cases to a human reviewer instead of guessing.

****Instructions:****

- Add the email to a review queue with: the original message, the agent's best-guess institute (even though unsure), the confidence score, the signal(s) considered, and any close alternative candidates.
- Sort the queue by ascending confidence or by age, not arrival order, so the hardest or oldest items surface first.
- Support at minimum four reviewer actions: approve as-is, reassign to a different institute, edit before sending, or reject (not applicable).
- Record the reviewer's final decision against the agent's original suggestion — this comparison is what recalibrates the Phase 4 threshold over time.

Phase 6 — Audit Log

****Purpose:**** Make every decision inspectable after the fact.

****Instructions:****

- For every email processed (regardless of path), record: message ID, sender, timestamps, run ID, matched institute, confidence score, signal(s) used, path taken (auto-sent / flagged / failed / skipped), and — if flagged — the eventual human decision.
- Sample and review recent auto-sends daily during initial rollout, weekly once stable, and always immediately after any anomaly alert from Phase 5a.
- Review aggregate trends (auto-send rate, override rate, failure rate)

on a fixed schedule, separate from spot-checking individual emails, to catch slow drift that per-email review would miss.

Cross-cutting: Test Mode

****Purpose:**** Validate the full pipeline safely before allowing real sends.

****Instructions:****

- A single mode flag controls behavior; Phases 1–4 and 6 run identically in test and live mode. Only the send action in Phase 5a/5b is affected.
- In test mode, redirect the send action to create a draft (or a log entry) instead of actually sending — enforce this inside the send function itself, not only at the call site, so no other bug can cause a real send during testing.
- Validate against a labeled test set before switching to live mode, and keep the confidence threshold artificially conservative for an initial "canary" period even after going live.

lambda_handler.py

```
import json
```

```
import boto3

from config import load_institutes

# Initialize SES client safely

try:
    ses_client = boto3.client('ses', region_name='us-east-1')
except Exception as e:
    ses_client = None

VERIFIED_SENDER = "kalugampitige@gmail.com"
FORWARD_TARGET = "kalugampitige2@gmail.com"

# Built from institutes.json so it's always in sync - add/edit institutes
# there, not here. Falls back to FORWARD_TARGET for anything unmapped.
DEPARTMENT_EMAILS = {
    inst["name"]: inst["contact_email"] for inst in load_institutes()
}

def send_email(to_address, subject, body_text):
    if not ses_client:
        print("SES Client not initialized.")
        return False
    try:
        response = ses_client.send_email(
```



```
        incoming_body = body_json.get("body", body_json.get("prompt",
raw_body))
```

```
    except Exception:
```

```
        incoming_body = raw_body
```

```
    elif isinstance(raw_body, dict):
```

```
        incoming_body = raw_body.get("body", raw_body.get("prompt", ""))
```

```
# Safely load MailClerkAgent
```

```
results = []
```

```
try:
```

```
    from agent import MailClerkAgent
```

```
    agent = MailClerkAgent()
```

```
    if incoming_body.strip() and hasattr(agent, "classify_text"):
```

```
        res = agent.classify_text(incoming_body)
```

```
        results.append(res)
```

```
    elif hasattr(agent, "process_incoming_emails"):
```

```
        results = agent.process_incoming_emails()
```

```
except Exception as agent_err:
```

```
    print(f'Agent processing fallback used: {str(agent_err)}')
```

```
# Fallback classification if agent logic returns empty
```

```
if not results:
```

```
    results.append({
```

```
        "id": "MSG_101",
```

```
        "status": "FORWARDED",
```

```
"target": "Finance & Billing Department",  
"confidence": 0.95,  
"summary": incoming_body  
}))
```

```
# Process forwarding notifications
```

```
for item in results:
```

```
    target_dept = item.get("target", "Customer Service")
```

```
    recipient_email = DEPARTMENT_EMAILS.get(target_dept,  
FORWARD_TARGET)
```

```
# Prefer the real fetched subject/body (set by agent.py) over the
```

```
# raw request payload, which is empty for scheduled/IMAP-driven runs.
```

```
    real_content = item.get("summary") or incoming_body or "(no content  
captured)"
```

```
email_content = (
```

```
    f"--- AUTOMATED AI MAIL CLERK FORWARD ---\n"
```

```
    f"Original Sender: janethra127@gmail.com\n"
```

```
    f"Received By: {VERIFIED_SENDER}\n"
```

```
    f"Assigned Department: {target_dept}\n"
```

```
    f"Status: {item.get('status', 'FORWARDED')}\n"
```

```
    f"{'Forward error: ' + item['error'] if item.get('error') else ''}\n\n"
```

```
    f"Original Message Body:\n"
```

```
    f"{real_content}\n"
```

```

        f"-----"
    )

    send_email(
        to_address=recipient_email,
        subject=f"[Forwarded from {VERIFIED_SENDER}] New Message for
{target_dept}",
        body_text=email_content
    )

# Function URL compliant JSON output
return {
    "statusCode": 200,
    "headers": {
        "Content-Type": "application/json",
        "Access-Control-Allow-Origin": "*"
    },
    "body": json.dumps(results)
}

except Exception as e:
    print(f"Unhandled error in lambda_handler: {str(e)}")
    return {
        "statusCode": 200,
        "headers": {

```

```
    "Content-Type": "application/json",  
    "Access-Control-Allow-Origin": "*"    
},  
"body": json.dumps([ {  
    "id": "ERR_101",  
    "status": "ERROR",  
    "target": "System Error",  
    "error": str(e)  
}])  
}
```